1. (3 Punkte) Schreibe eine Funktion `lineareSuche(a, x)`, die den Index des ersten Vorkommens von `x` in der Liste `a` zurückgibt, oder -1, wenn `x` nicht vorkommt.

Lösung:

```
def lineareSuche(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1
```

2. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
def binaereSuche(a, x):
    L = 0
    R = len(a)-1
    while L <= R:
        mid = (L+R)//2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            L = mid + 1
        else:
            R = mid - 1
    return -1

x = 32
a = [4, 9, 13, 33, 35, 41, 46, 47, 49, 54, 58, 80, 89, 94, 97]
```

Lösung:

x = 32

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	9	13	33	35	41	46	47	49	54	58	80	89	94	97

Schleife	L	R	mid	return
0	0	14		
1	0	6	7	
2	0	2	3	
3	2	2	1	
4	3	2	2	
				-1

3. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
x = 49
a = [4, 7, 10, 11, 13, 17, 21, 24, 27, 29, 30, 32, 40, 45]
```

Lösung:

x = 49

0	1	2	3	4	5	6	7	8	9	10	11	12	13
4	7	10	11	13	17	21	24	27	29	30	32	40	45

Schleife	L	R	mid	return
0	0	13		
1	7	13	6	
2	11	13	10	
3	13	13	12	
4	14	13	13	
				-1

4. (4 Punkte) Erstelle nach dem Verfahren aus dem Unterricht eine Tabelle für die Variablen L, R, mid und den Rückgabewert wenn die binäre Suche auf die Liste a und das gesuchte Element x angewendet wird.

```
x = 13
a = [4, 7, 10, 11, 13, 17, 21, 24]
```

Lösung:

x = 13

	0	1	2	3	4	5	6	7
	4	7	10	11	13	17	21	24

Schleife	L	R	mid	return
0	0	7		
1	4	7	3	
2	4	4	5	
3	4	4	4	4

5. (3 Punkte) (a) Eine sortierte Liste hat 1000, 1 Million bzw. 1 Milliarde Elemente. Wie viele Schleifendurchläufe braucht die binäre Suche höchstens? Begründe kurz.

   (b) Wie viele Elemente muss die lineare Suche bei einer Milliarde Elementen höchstens ansehen?

   (c) Was liefert die binäre Suche aus dem Unterricht für die Liste `[5, 1, 4, 2, 3]` und `x = 1`? Warum?

Lösung:

a) 10, 20 bzw. 30. Jeder Durchlauf halbiert den Suchbereich, und 1000 $\approx 2^{10}$, 1 Million $\approx 2^{20}$, 1 Milliarde $\approx 2^{30}$.

b) 1 Milliarde, wenn `x` am Ende steht oder nicht in der Liste ist.

c) `-1`, obwohl 1 in der Liste steht: `mid = 2`, `a[2] = 4 > 1`, also wird links weitergesucht; `mid = 0`, `a[0] = 5 > 1`, dann ist `R = -1`. Die Liste ist nicht sortiert, die binäre Suche funktioniert nur für sortierte Listen.

6. (5 Punkte) Schreibe eine Funktion `binaereSuche(a, x)`, die in der sortierten Liste `a` den Wert `x` sucht. Sie gibt den Index von `x` zurück, oder -1, wenn `x` nicht vorkommt.

Lösung:

```
def binaereSuche(a, x):
    L = 0
    R = len(a) - 1
    while L <= R:
        mid = L + (R - L) // 2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            L = mid + 1
        else:
            R = mid - 1
    return -1
```

7. (3 Punkte) In einem Online-Shop sollen Artikel über ihre Nummer gefunden werden.
   (a) Die Artikel stehen in einer unsortierten Liste. Welches Suchverfahren können wir benutzen?
   (b) Wie viele Schritte braucht es bei 1 Million Artikeln im schlechtesten Fall?
   (c) Wie viele Schritte braucht die binäre Suche, wenn die Liste sortiert ist?

Lösung: a) nur die lineare Suche
b) 1 Million
c) etwa 20, denn $2^{20} \approx 10^6$